

DESIGN PATTERNS

Creational & Behavioral Design Patterns in C#

Factory Method Pattern

Defines an interface for creating objects, but lets subclasses decide which class to instantiate. Promotes loose coupling between creators and concrete products.

Product Interface & Implementations

```
public interface INotification
{
    void NotifyUser();
}

public class EmailNotification : INotification
{
    public void NotifyUser() => Console.WriteLine("Sending Email");
}

public class SmsNotification : INotification
{
    public void NotifyUser() => Console.WriteLine("Sending SMS");
}
```

Factory & Usage

```
public abstract class NotificationFactory
{
    public abstract INotification CreateNotification();
}

public class EmailFactory : NotificationFactory
{
    public override INotification CreateNotification() => new EmailNotification();
}

// Usage
NotificationFactory factory = new EmailFactory();
INotification notification = factory.CreateNotification();
notification.NotifyUser();
```

Abstract Factory Pattern

Provides an interface for creating **families of related or dependent objects** without specifying their concrete classes. Ideal for UI toolkits, platform-specific components, and themed systems.

Product Families

```
public interface IButton { void Paint(); }
public interface ICheckbox { void Paint(); }

public class WinButton : IButton
{
    public void Paint() => Console.WriteLine("Windows Button");
}

public class WinCheckbox : ICheckbox
{
    public void Paint() => Console.WriteLine("Windows Checkbox");
}

public class MacButton : IButton
{
    public void Paint() => Console.WriteLine("Mac Button");
}

public class MacCheckbox : ICheckbox
{
    public void Paint() => Console.WriteLine("Mac Checkbox");
}
```

Abstract Factory & Usage

```
public interface IGUIFactory
{
    IButton CreateButton();
    ICheckbox CreateCheckbox();
}

public class WinFactory: IGUIFactory
{
    public IButton CreateButton() => new WinButton();
    public ICheckbox CreateCheckbox() => new WinCheckbox();
}

// Usage
IGUIFactory factory = new WinFactory();
IButton btn = factory.CreateButton();
btn.Paint();
```

Builder Pattern

Separates the construction of a complex object from its representation. Uses a **fluent interface** to configure properties step-by-step before building the final immutable object.

```
public class Computer
{
    public string CPU { get; }
    public int RAM { get; }
    public int Storage { get; }
    public bool HasGPU { get; }

    private Computer(Builder builder)
    {
        CPU = builder.CPU;
        RAM = builder.RAM;
        Storage = builder.Storage;
        HasGPU = builder.HasGPU;
    }

    public class Builder
    {
        public string CPU { get; private set; }
        public int RAM { get; private set; }
        public int Storage { get; private set; }
        public bool HasGPU { get; private set; }

        public Builder WithCPU(string cpu)
        {
            CPU = cpu;
            return this;
        }

        public Builder WithRAM(int ram)
        {
            RAM = ram;
            return this;
        }

        public Builder WithStorage(int storage)
        {
            Storage = storage;
            return this;
        }

        public Builder WithGPU(bool gpu)
        {
            HasGPU = gpu;
            return this;
        }

        public Computer Build() => new Computer(this);
    }
}
```

Fluent Usage

```
// Usage
var pc = new Computer.Builder()
    .WithCPU("Intel i9")
    .WithRAM(32)
    .WithStorage(1000)
    .WithGPU(true)
    .Build();
```

- ❏ The Builder pattern eliminates telescoping constructors and makes object creation readable and maintainable.

Observer Pattern

Defines a one-to-many dependency between objects. When one object changes state, all its dependents are notified automatically.

```
public interface IObserver
{
    void Update(string message);
}

public class NewsChannel : IObserver
{
    public void Update(string message) => Console.WriteLine($"Breaking: {message}");
}

public class NewsAgency
{
    private readonly List<IObserver> _observers = new();

    public void Subscribe(IObserver observer) => _observers.Add(observer);

    public void Unsubscribe(IObserver observer) => _observers.Remove(observer);

    public void SetNews(string news)
    {
        foreach (var observer in _observers)
            observer.Update(news);
    }
}

// Usage
var agency = new NewsAgency();
var channel1 = new NewsChannel();
var channel2 = new NewsChannel();

agency.Subscribe(channel1);
agency.Subscribe(channel2);

agency.SetNews("The Observer Pattern works!");
```

Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can swap strategies at runtime without modifying the context class.

Strategy Interface & Implementations

```
public interface IPaymentStrategy
{
    void Pay(int amount);
}

public class CreditCardPayment : IPaymentStrategy
{
    public void Pay(int amount) => Console.WriteLine($"Paid {amount} via Credit Card");
}

public class PayPalPayment : IPaymentStrategy
{
    public void Pay(int amount) => Console.WriteLine($"Paid {amount} via PayPal");
}
```

Context & Usage

```
public class ShoppingCart
{
    private IPaymentStrategy _strategy;

    public void SetPaymentStrategy(IPaymentStrategy strategy) => _strategy = strategy;

    public void Checkout(int amount) => _strategy.Pay(amount);
}

// Usage
var cart = new ShoppingCart();

cart.SetPaymentStrategy(new CreditCardPayment());
cart.Checkout(100);

cart.SetPaymentStrategy(new PayPalPayment());
cart.Checkout(200);
```


SUMMARY

Pattern Comparison

A quick reference to help you choose the right pattern for your scenario.

Pattern	Type	Use Case
Factory Method	Creational	Delegate creation to subclasses
Abstract Factory	Creational	Families of related objects
Builder	Creational	Step-by-step complex object construction
Observer	Behavioral	Event subscription / Pub-Sub
Strategy	Behavioral	Interchangeable algorithms

Creational patterns focus on **how objects are created**, while behavioral patterns define **how objects communicate and delegate responsibilities**.